

Lab Experiment: 10

Assignment 1st: AVL Tree Implementation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define the structure for an AVL Tree Node
```

```
struct TreeNode {
```

```
    int data;
```

```
    struct TreeNode* left;
```

```
    struct TreeNode* right;
```

```
    int height;
```

```
};
```

```
// Helper function to get the height of a node
```

```
int height(struct TreeNode* node) {
```

```
    return (node == NULL) ? 0 : node->height;
```

```
}
```

```
// Helper function to calculate the balance factor
```

```
int getBalance(struct TreeNode* node) {
```

```
    return (node == NULL) ? 0 : height(node->left) - height(node->right);
```

```
}
```

```
// Create a new AVL Tree node
```

```
struct TreeNode* createNode(int value) {
```

```
    struct TreeNode* newNode = (struct TreeNode*)malloc(sizeof(struct TreeNode));
```

```
    newNode->data = value;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    newNode->height = 1; // New node is initially at height 1
```

```
    return newNode;
```

```

}

// Right Rotate (for LL imbalance)
struct TreeNode* rightRotate(struct TreeNode* y) {
    struct TreeNode* x = y->left;
    struct TreeNode* T2 = x->right;

    // Perform rotation
    x->right = y;
    y->left = T2;

    // Update heights
    y->height = 1 + (height(y->left) > height(y->right) ? height(y->left) : height(y->right));
    x->height = 1 + (height(x->left) > height(x->right) ? height(x->left) : height(x->right));

    return x;
}

// Left Rotate (for RR imbalance)
struct TreeNode* leftRotate(struct TreeNode* x) {
    struct TreeNode* y = x->right;
    struct TreeNode* T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

    // Update heights
    x->height = 1 + (height(x->left) > height(x->right) ? height(x->left) : height(x->right));
    y->height = 1 + (height(y->left) > height(y->right) ? height(y->left) : height(y->right));

    return y;
}

// Insert a node and balance the tree if needed

```

```

struct TreeNode* insertNode(struct TreeNode* node, int value) {

    // 1. Perform the normal BST insertion

    if (node == NULL)

        return createNode(value);

    if (value < node->data)

        node->left = insertNode(node->left, value);

    else if (value > node->data)

        node->right = insertNode(node->right, value);

    else

        return node; // No duplicates allowed

    // 2. Update height of this ancestor node

    node->height = 1 + (height(node->left) > height(node->right) ? height(node->left) : height(node->right));

    // 3. Get the balance factor to check if this node became unbalanced

    int balance = getBalance(node);

    // Left Left Case

    if (balance > 1 && value < node->left->data)

        return rightRotate(node);

    // Right Right Case

    if (balance < -1 && value > node->right->data)

        return leftRotate(node);

    // Left Right Case

    if (balance > 1 && value > node->left->data) {

        node->left = leftRotate(node->left);

```

```

        return rightRotate(node);
    }

    // Right Left Case
    if (balance < -1 && value < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

// Find the node with minimum value in the subtree
struct TreeNode* minValueNode(struct TreeNode* node) {
    struct TreeNode* current = node;
    while (current->left != NULL)
        current = current->left;
    return current;
}

// Delete a node and balance the tree if needed
struct TreeNode* deleteNode(struct TreeNode* root, int value) {
    // 1. Perform the normal BST deletion
    if (root == NULL)
        return root;

    if (value < root->data)
        root->left = deleteNode(root->left, value);
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
    else {
        // Node with only one child or no child

```

```

if ((root->left == NULL) || (root->right == NULL)) {
    struct TreeNode* temp = root->left ? root->left : root->right;
    if (temp == NULL) {
        temp = root;
        root = NULL;
    } else
        *root = *temp;
    free(temp);
} else {
    // Node with two children
    struct TreeNode* temp = minValueNode(root->right);
    root->data = temp->data;
    root->right = deleteNode(root->right, temp->data);
}
}

// If the tree had only one node then return
if (root == NULL)
    return root;

// 2. Update height
root->height = 1 + (height(root->left) > height(root->right) ? height(root->left) : height(root->right));

// 3. Get balance factor to check if this node became unbalanced
int balance = getBalance(root);

// Left Left Case
if (balance > 1 && getBalance(root->left) >= 0)
    return rightRotate(root);

```

```

// Left Right Case
if (balance > 1 && getBalance(root->left) < 0) {
    root->left = leftRotate(root->left);
    return rightRotate(root);
}

// Right Right Case
if (balance < -1 && getBalance(root->right) <= 0)
    return leftRotate(root);

// Right Left Case
if (balance < -1 && getBalance(root->right) > 0) {
    root->right = rightRotate(root->right);
    return leftRotate(root);
}

return root;
}

// In-order traversal
void inOrder(struct TreeNode* root) {
    if (root != NULL) {
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}

int main() {

```

```
struct TreeNode* root = NULL;

int choice, value;

do {

    printf("\nAVL Tree Menu:\n");

    printf("1. Insert\n");

    printf("2. Delete\n");

    printf("3. Display (In-order Traversal)\n");

    printf("4. Exit\n");

    printf("Enter choice: ");

    scanf("%d", &choice);


    switch (choice) {

        case 1:

            printf("Enter value to insert: ");

            scanf("%d", &value);

            root = insertNode(root, value);

            break;

        case 2:

            printf("Enter value to delete: ");

            scanf("%d", &value);

            root = deleteNode(root, value);

            break;

        case 3:

            printf("In-order traversal: ");

            inOrder(root);

            printf("\n");

            break;

        case 4:

            printf("Exiting program.\n");
```

```

        break;

    default:

        printf("Invalid choice.\n");

    }

} while (choice != 4);

return 0;

}

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 1
Enter value to insert: 7

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 1
Enter value to insert: 3

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 1
Enter value to insert: 5

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 1
Enter value to insert: 11

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 3
In-order traversal: 3 5 7 11

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 2
Enter value to delete: 7

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit
Enter choice: 3
In-order traversal: 3 5 11

```

```

AVL Tree Menu:
1. Insert
2. Delete
3. Display (In-order Traversal)
4. Exit

```

Assignment 2nd: Heap Sort Implementation

```
#include <stdio.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to heapify a subtree with root at given index `i`
void heapify(int arr[], int n, int i) { // `n` is the size of the heap
    int largest = i;    // Initialize largest as root
    int left = 2 * i + 1; // Left child index
    int right = 2 * i + 2; // Right child index

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger than largest so far
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}
```

```

}

// Function to build a max heap from the input array
void buildMaxHeap(int arr[], int n) {
    // Start from the last non-leaf node and heapify each node
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
}

// Function to perform heap sort
void heapSort(int arr[], int n) {
    buildMaxHeap(arr, n);
    for (int i = n - 1; i > 0; i--) {
        // Move current root to end (largest element)
        swap(&arr[0], &arr[i]);

        // Reduce the heap size and heapify the root
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {11, 13, 6, 17, 8};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

```

```

// Perform heap sort
heapSort(arr, n);

printf("Sorted array (Heap Sort): ");
printArray(arr, n);
return 0;
}

```

```

Original array: 11 13 6 17 8
Sorted array (Heap Sort): 6 8 11 13 17

=== Code Execution Successful ===

```

Assignment 3rd: Priority Queue Using Heap

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

typedef struct {
    int data;    // The value of the element
    int priority; // The priority of the element
} PriorityQueueElement;

```

```
// Priority Queue structure with array-based max heap
```

```

typedef struct {
    PriorityQueueElement* elements; // Array of elements in the priority queue
    int size;                       // Current size of the priority queue
    int capacity;                   // Maximum capacity of the priority queue
} PriorityQueue;

```

// Function to create a new priority queue with a given capacity

```
PriorityQueue* createPriorityQueue(int capacity) {  
    PriorityQueue* pq = (PriorityQueue*)malloc(sizeof(PriorityQueue));  
    pq->elements = (PriorityQueueElement*)malloc(capacity * sizeof(PriorityQueueElement));  
    pq->size = 0;  
    pq->capacity = capacity;  
    return pq;  
}
```

```
void swap(PriorityQueueElement* a, PriorityQueueElement* b) {  
    PriorityQueueElement temp = *a;  
    *a = *b;  
    *b = temp;  
}
```

```
void heapifyUp(PriorityQueue* pq, int index) {  
    int parent = (index - 1) / 2;  
    if (index > 0 && pq->elements[index].priority > pq->elements[parent].priority) {  
        swap(&pq->elements[index], &pq->elements[parent]);  
        heapifyUp(pq, parent);  
    }  
}
```

// Function to insert an element with a priority into the priority queue

```
void insert(PriorityQueue* pq, int data, int priority) {  
    if (pq->size == pq->capacity) {  
        printf("Priority Queue is full!\n");  
        return;  
    }  
}
```

```

// Insert the new element at the end
pq->elements[pq->size].data = data;
pq->elements[pq->size].priority = priority;
pq->size++;
// Heapify up to maintain max heap property
heapifyUp(pq, pq->size - 1);
}

void heapifyDown(PriorityQueue* pq, int index) {
    int largest = index;
    int left = 2 * index + 1;
    int right = 2 * index + 2;

    if (left < pq->size && pq->elements[left].priority > pq->elements[largest].priority)
        largest = left;

    if (right < pq->size && pq->elements[right].priority > pq->elements[largest].priority)
        largest = right;

    if (largest != index) {
        swap(&pq->elements[index], &pq->elements[largest]);
        heapifyDown(pq, largest);
    }
}

// Function to remove the element with the highest priority from the queue
PriorityQueueElement removeMax(PriorityQueue* pq) {
    if (pq->size == 0) {
        printf("Priority Queue is empty!\n");
    }
}

```

```

        return (PriorityQueueElement){-1, -1}; // Indicate an empty queue
    }

    // Root of the heap is the highest priority element
    PriorityQueueElement maxElement = pq->elements[0];

    // Move the last element to the root and reduce the size
    pq->elements[0] = pq->elements[pq->size - 1];
    pq->size--;

    // Heapify down from the root to maintain max heap property
    heapifyDown(pq, 0);

    return maxElement;
}

// Function to print all elements in the priority queue (for demonstration)
void printPriorityQueue(PriorityQueue* pq) {
    printf("Priority Queue elements:\n");
    for (int i = 0; i < pq->size; i++) {
        printf("Data: %d, Priority: %d\n", pq->elements[i].data, pq->elements[i].priority);
    }
}

int main() {
    int capacity = 10;

    PriorityQueue* pq = createPriorityQueue(capacity);

    insert(pq, 10, 1);
    insert(pq, 20, 3);
    insert(pq, 30, 2);
    insert(pq, 40, 5);

    printf("Initial Priority Queue:\n");

```

```

printPriorityQueue(pq);

PriorityQueueElement maxElement = removeMax(pq);

printf("\nRemoved element with highest priority: Data = %d, Priority = %d\n", maxElement.data,
maxElement.priority);

printf("\nPriority Queue after removal:\n");

printPriorityQueue(pq);

// Free memory
free(pq->elements);
free(pq);

return 0;
}

```

```

Initial Priority Queue:
Priority Queue elements:
Data: 40, Priority: 5
Data: 20, Priority: 3
Data: 30, Priority: 2
Data: 10, Priority: 1

Removed element with highest priority: Data = 40, Priority = 5

Priority Queue after removal:
Priority Queue elements:
Data: 20, Priority: 3
Data: 10, Priority: 1
Data: 30, Priority: 2

=== Code Execution Successful ===

```